



《人工智能与Python程序设计》——卷积神经网络



人工智能与Python程序设计 教研组



提纲



卷积神经网络

- □ 单个卷积层
- □ 池化层
- □ 卷积神经网络
- □ 经典卷积神经网络



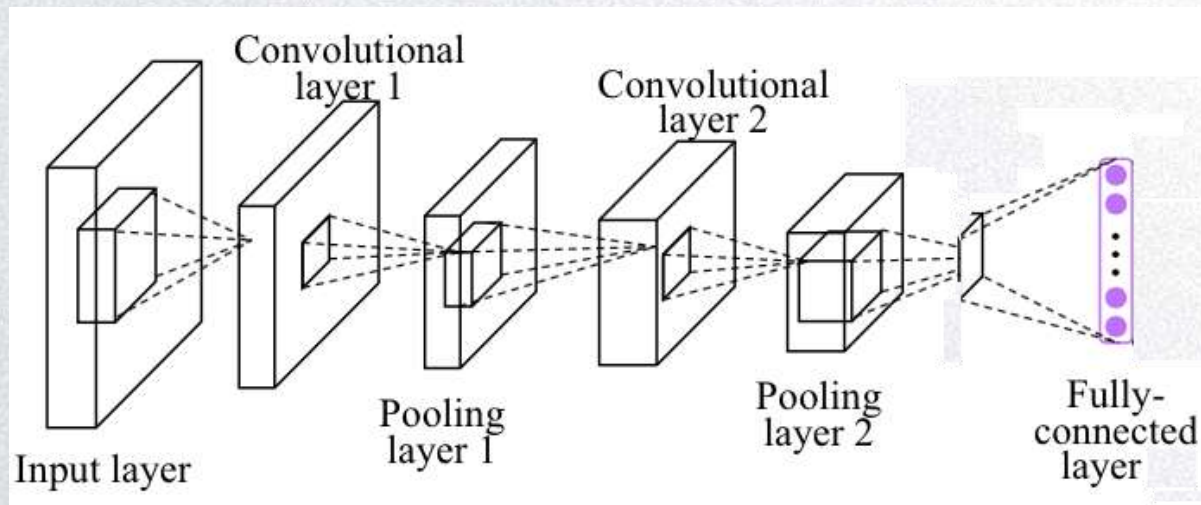
手写数字识别

- $3 \times 32 \times 32$ 的RGB图片中含有某个数字，识别它是从0-9这10个数字中的哪一个
- 分类问题



卷积神经网络

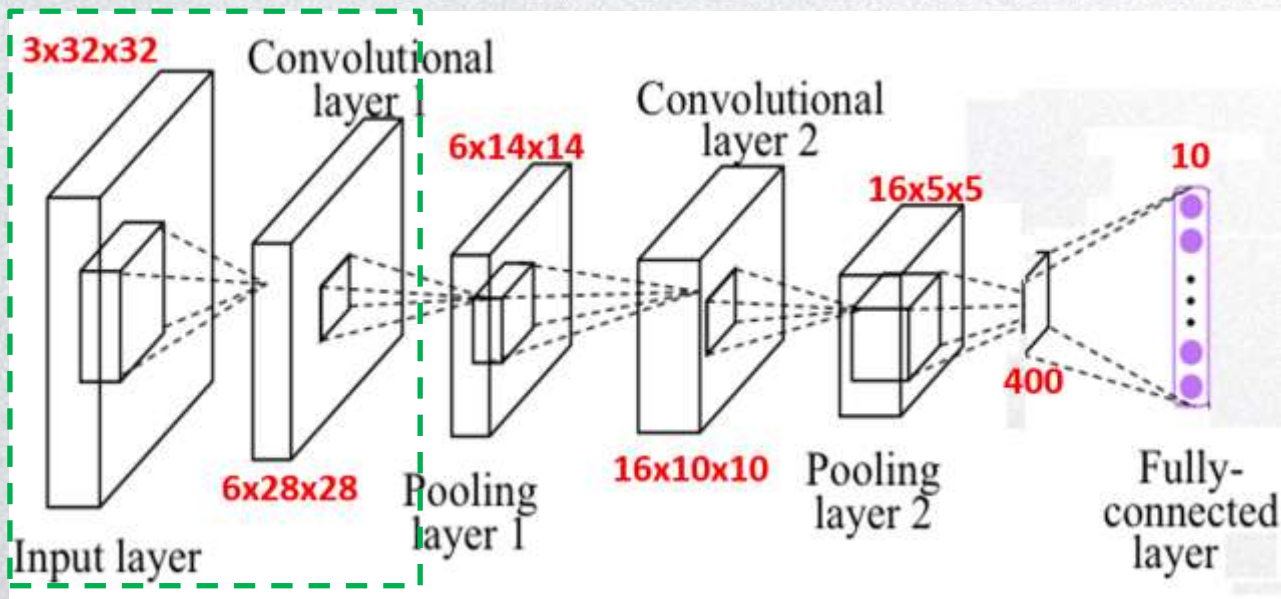
- 类似LeNet-5 (Yann LeCun创建)



卷积神经网络

卷积层，卷积完之后的尺寸为 $3 \times 28 \times 28$

- 输入是 $3 \times 32 \times 32$ 的矩阵，第一层使用滤波器大小为 5×5 ，步幅是1，padding是0，滤波器个数为6，那么输出为 $6 \times 28 \times 28$ 。将这层标记为CONV1，增加偏差，应用非线性函数（如ReLU）后输出结果。



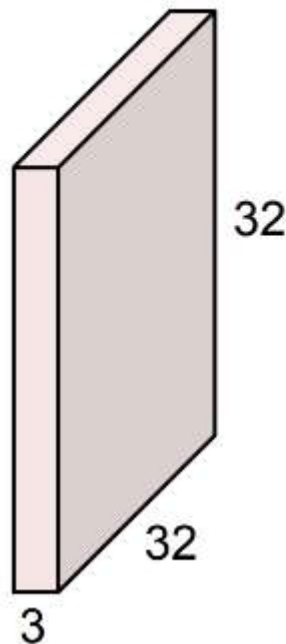
默认一个卷积跟着一个池化，一个卷积后边也有一个非线性变化。

单个卷积层

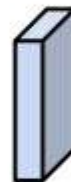
- 输入图片大小是 $3 \times 32 \times 32$
- 用 $3 \times 5 \times 5$ 的卷积核

- 输出: $(32+0-5) / 1 + 1$

32x32x3 image

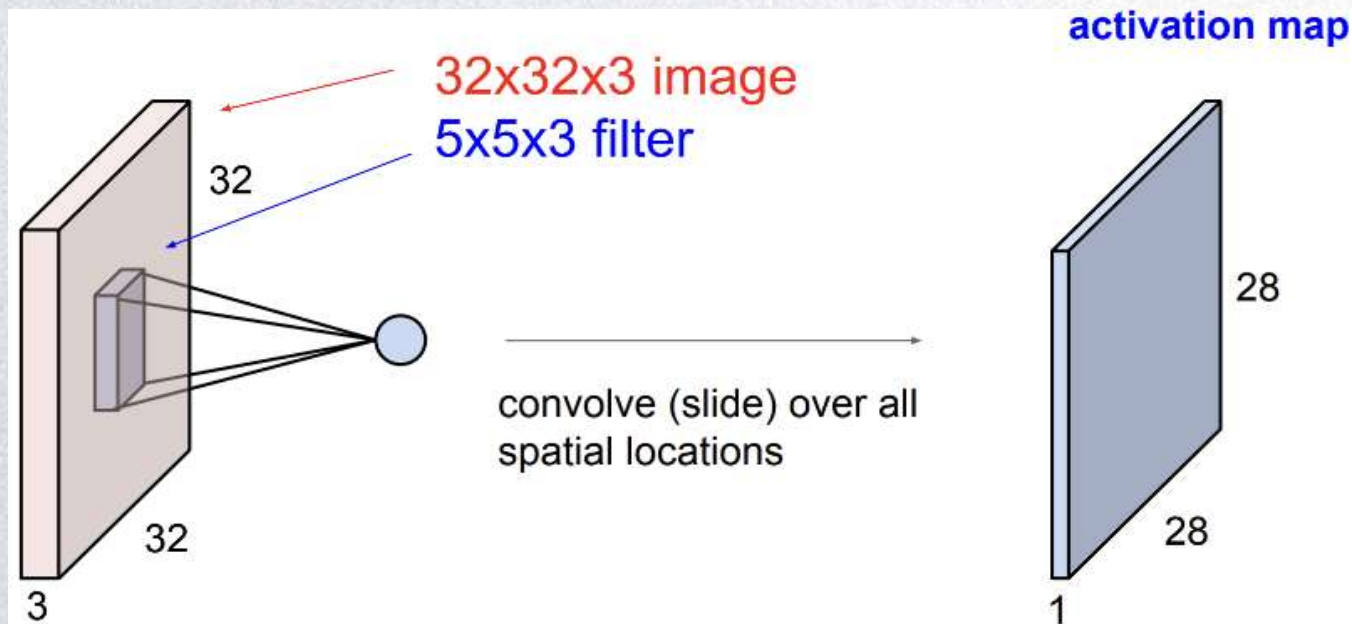


5x5x3 filter



单个卷积层

- 单个卷积核得到特征图



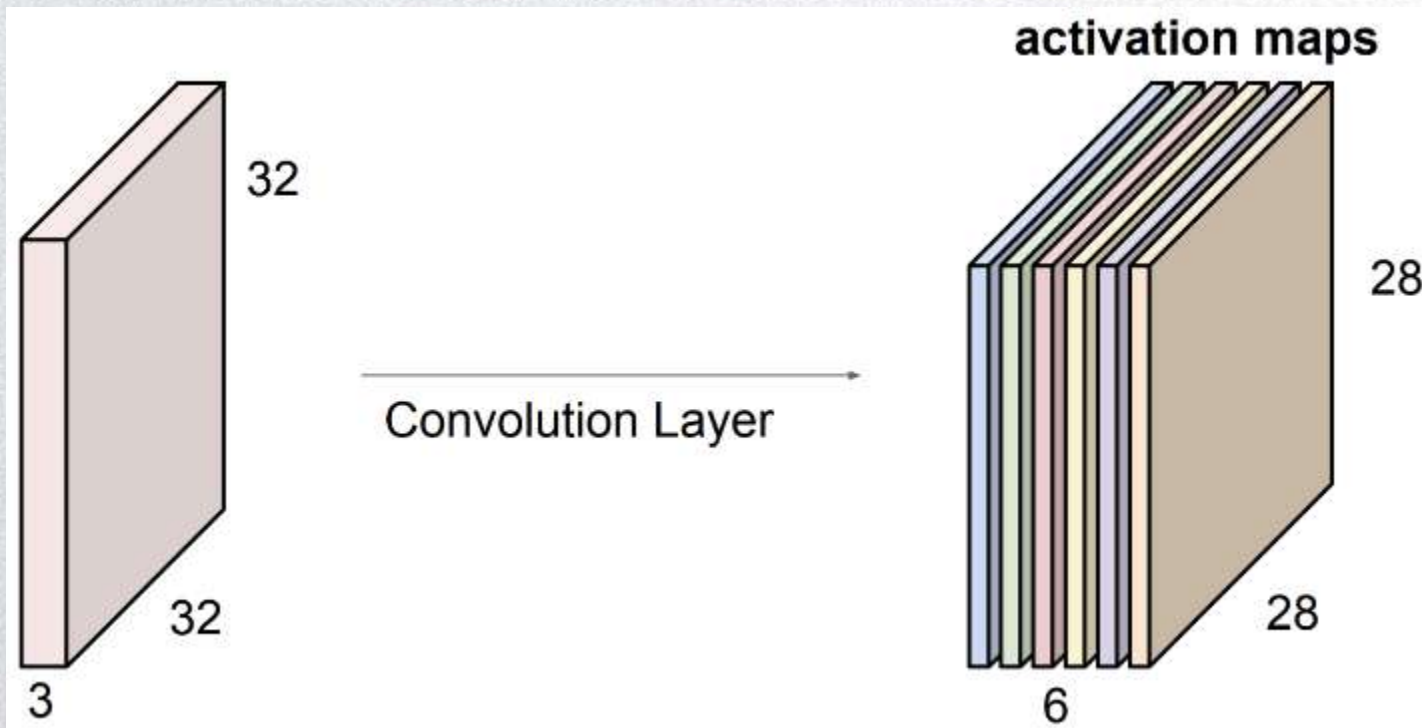
单个卷积层

- 每个卷积核得到一个特征图



单个卷积层

- 总共有6个卷积核





单个卷积层

- 前向传播（在每个局部） $z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$
- 非线性函数： $a^{[1]} = g(z^{[1]})$
- 滤波器用变量 $W^{[1]}$ 表示。

卷积可以理解为一个特殊的线性变换

增加bias增强拟合的能力

一个输出通道只有一个统一的bias

- 应用激活函数ReLU，得到的 $6 \times 28 \times 28$ 矩阵成为神经网络的下一层的输入。
- 通过这个过程把一个 $3 \times 32 \times 32$ 维度的输入图像变换为一个 $6 \times 28 \times 28$ 维度的特征图，这就是卷积神经网络的一层



Pytorch卷积层

- Pytorch实现这些只需要一行代码

在这里卷积核是我可以优化的一个参数

- `nn.Conv2d(self, in_channels, out_channels, kernel_size, stride=1, padding=0, dilation=1, groups=1, bias=True)`
 - `in_channel`: 输入数据的通道数;
 - `out_channel`: 输出数据的通道数;
 - `kennel_size`: 卷积核大小, 可以是int, 或tuple;
 - `stride`: 步长;
 - `padding`



Pytorch卷积层



```
1  import torch
2  import torch.nn as nn
3  import numpy as np
4  from PIL import Image
5  from torchvision import transforms
6  import torch.nn.functional as F
7
8  image = Image.open('5.jpg').convert('RGB')
9  input = transforms.ToTensor()(image).unsqueeze(0)
10 print(input.shape)
11 # Conv-1
12 Conv1 = nn.Conv2d(in_channels=3,out_channels=6,kernel_size=5,stride=1,padding=0)
13 Nf1 = nn.ReLU()
14 output1 = Nf1(Conv1(input))
15 print(output1.shape)
```

```
torch.Size([1, 3, 32, 32])
torch.Size([1, 6, 28, 28])
```



提纲

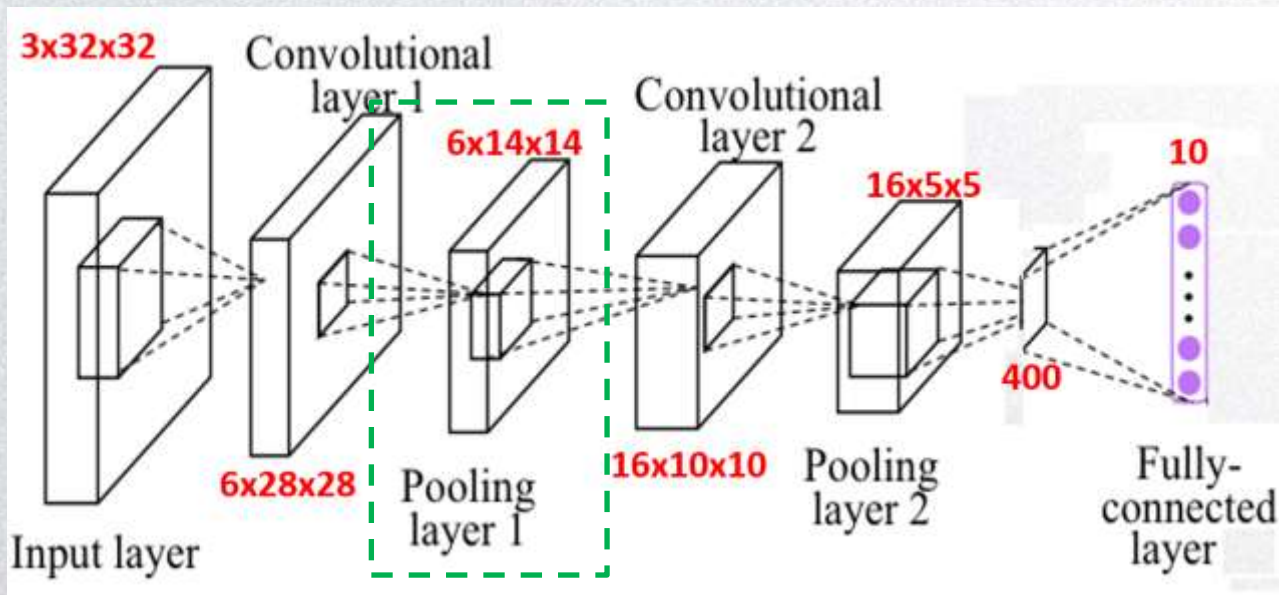


卷积神经网络

- □ 单个卷积层
- □ 池化层
- □ 卷积神经网络
- □ 经典卷积神经网络

卷积神经网络

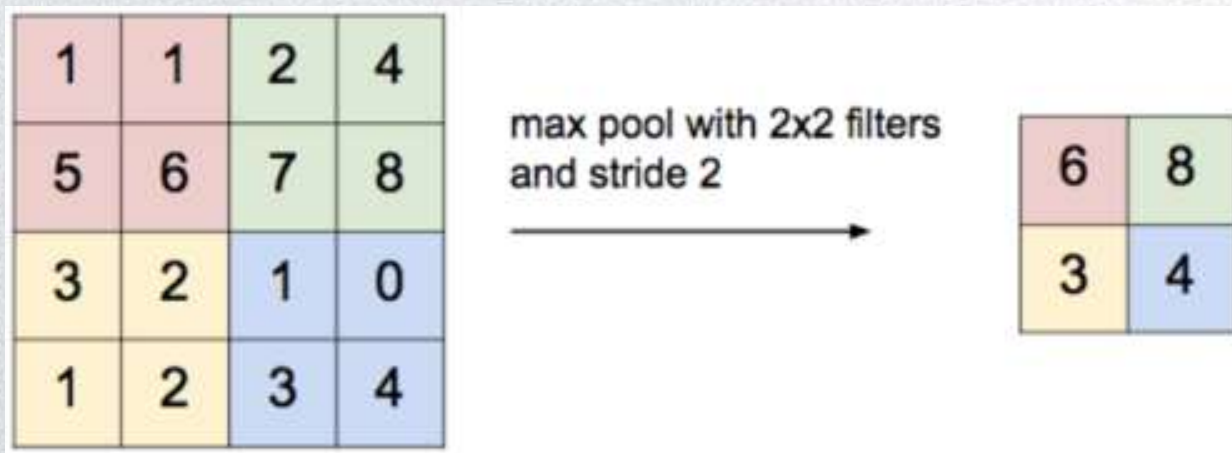
- 在第一个卷积层之后，构建一个池化层，这里选最大池化，过滤器为 2×2 ，步幅为2，padding为0。最终输出为 $6 \times 14 \times 14$ ，将该层标记为**POOL1**。



池化层

- 特征图展开成向量可能维度很大
- 池化：缩减模型的大小；提高计算速度；提高所提取特征的鲁棒性
- Max pooling:

仅仅提取每个区域的最大数值



最大池化

- 最大池化

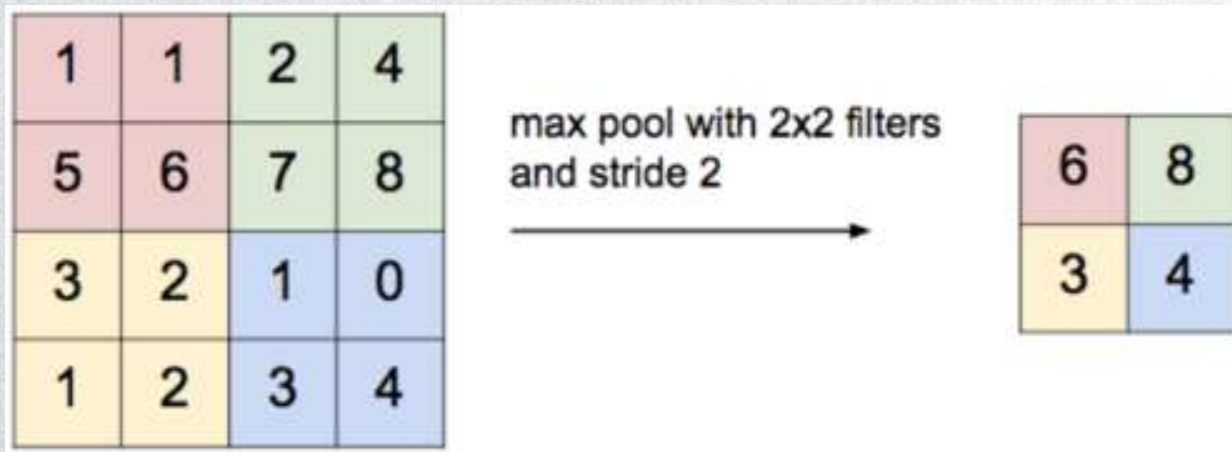
3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

3.0	3.0	3.0
3.0	3.0	3.0
3.0	2.0	3.0

输出结果也和卷积结果想用

最大池化

- 输入可能是神经网络中某一层的非激活值，可以看作是某些特征的集合。数字大意味着可能探测到了某些特定的特征。
- 如果在过滤器中提取到某个特征，那么保留其最大值。如果没有提取到这个特征，可能在左下象限中不存在这个特征，那么其中的最大值也还是很小。





最大池化

- 最大池化有一组超参数，但并没有参数需要学习。一旦确定了 f 和 s ，它就是一个固定运算，梯度下降无需改变任何值。
- 计算卷积层输出大小的公式同样适用于最大池化。
- 如果输入是三维的，那么输出也是三维的。例如，输入是 $nc \times 5 \times 5$ ， $f=3$ ， $s=1$ ，那么输出是 $nc \times 3 \times 3$ 。计算最大池化的方法就是分别对每个通道执行刚刚的计算过程。

平均池化

- 选取的不是每个过滤器的最大值，而是平均值
- $f=3, s=1$

3	3	2	1	0
0	0	1	3	1
3	1	2	2	3
2	0	0	2	2
2	0	0	0	1

1.7	1.7	1.7
1.0	1.2	1.8
1.1	0.8	1.3



Pytorch池化层

- `nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1, return_indices=False, ceil_mode=False)`
 - `kernel_size`: 卷积核大小, 可以是int, 或tuple;
 - `stride`: 步长;
 - `padding`
- `nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False, count_include_pad=True)`



Pytorch池化层

```
1  import torch
2  import torch.nn as nn
3
4  x = torch.tensor([[3., 3., 2., 1., 0.], [0., 0., 1., 3., 1.]
5  x = x.view(1,5,5)
6  MP = nn.MaxPool2d(kernel_size=3, stride=1)
7  y = MP(x)
8  print(y)
9
10 MP2 = nn.AvgPool2d(3,1)
11 y = MP2(x)
12 print(y)
```

```
tensor([[[[3., 3., 3.],
          [3., 3., 3.],
          [3., 2., 3.]]]])
tensor([[[[1.6667, 1.6667, 1.6667],
          [1.0000, 1.2222, 1.7778],
          [1.1111, 0.7778, 1.3333]]]])
```



Pytorch池化层

池化层不一定是一个正方形，也可以是一个小矩形

```
a = torch.randn(3,5,10)
MP3 = nn.MaxPool2d((5,1))
c = MP3(a)
print(c.shape)

x = torch.rand(1,3,7,7)
MP4 = nn.AvgPool2d(kernel_size=2, stride=2)
print(MP4.forward(x).shape)
```

```
torch.Size([3, 1, 10])
torch.Size([1, 3, 3, 3])
```



池化层总结

- 池化的超参数包括过滤器大小 f 、步幅 s 、池化方式（最大池化和平均池化），常用的参数值为 $f=2$ ， $s=2$ ，效果相当于高度和宽度缩减一半。
- 也可以增加表示padding的其他超参数，但很少这么用。
- 输入通道与输出通道个数相同，因为对每个通道都做了池化。
- 池化过程中没有需要学习的参数，只有这些超参数，可以是手动设置的，也可以是通过交叉验证设置的。



卷积神经网络

- 构建一个池化层，这里选最大池化，过滤器为 2×2 ，步幅为2，padding为0。最终输出为 $6 \times 14 \times 14$ ，将该输出标记为**POOL1**。
- 在计算神经网络有多少层时，通常只统计具有权重和参数的层。这里，把CONV1和POOL1共同作为一个卷积，并标记为Layer1。

```
16 # Pool-1
17 Pool1 = nn.MaxPool2d((2,2),stride=2,padding=0)
18 output1 = Pool1(output1)
19 print(output1.shape)
```

```
torch.Size([1, 6, 14, 14])
```



层的划分

- 在卷积神经网络文献中，卷积有两种分类，这与所谓层的划分有关。
- 一类卷积是一个卷积层和一个池化层一起作为一层，这就是神经网络的**Layer1**。卷积层默认还会包含一次非线性变化，一次卷积对应一次池化
- 另一类卷积是把卷积层作为一层，而池化层单独作为一层。
- 在计算神经网络有多少层时，通常只统计具有权重和参数的层。因为池化层没有权重和参数，只有一些超参数。
- 在阅读网络文章或研究报告时，可能会看到卷积层和池化层各为一层的情况，这只是两种不同的标记术语。这里我们用**CONV1**和**POOL1**来标记，两者都是神经网络**Layer1**的一部分。



提纲



卷积神经网络

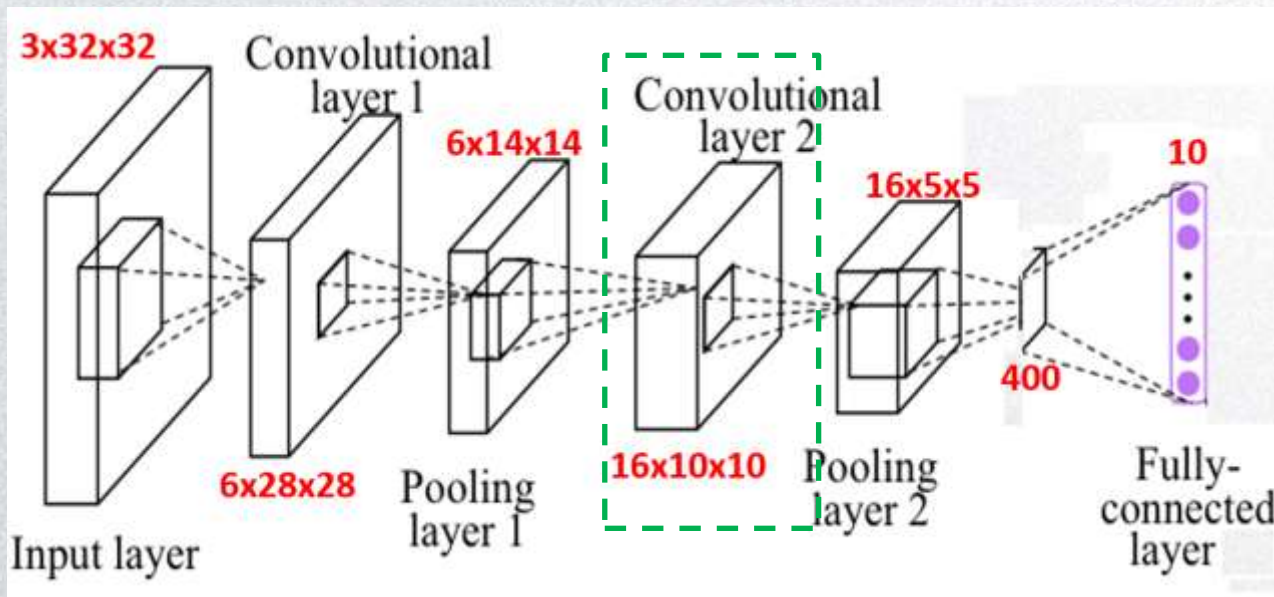
- □ 单个卷积层
- □ 池化层
- □ 卷积神经网络
- □ 经典卷积神经网络



卷积神经网络

- 再构建一个卷积层，过滤器大小为 5×5 ，步幅为1，这次用16个过滤器，最后输出一个 $16 \times 10 \times 10$ 的矩阵，标记为CONV2。

过滤器滤波器和卷积核在这里被认为是一个东西



参数的个数为：
一个卷积核中的参数（不含bias）： $6 \times 5 \times 5$
一个卷积核加一个bias： $6 \times 5 \times 5 + 1$
输出通道为16，则有卷积核成16



练习：CONV2

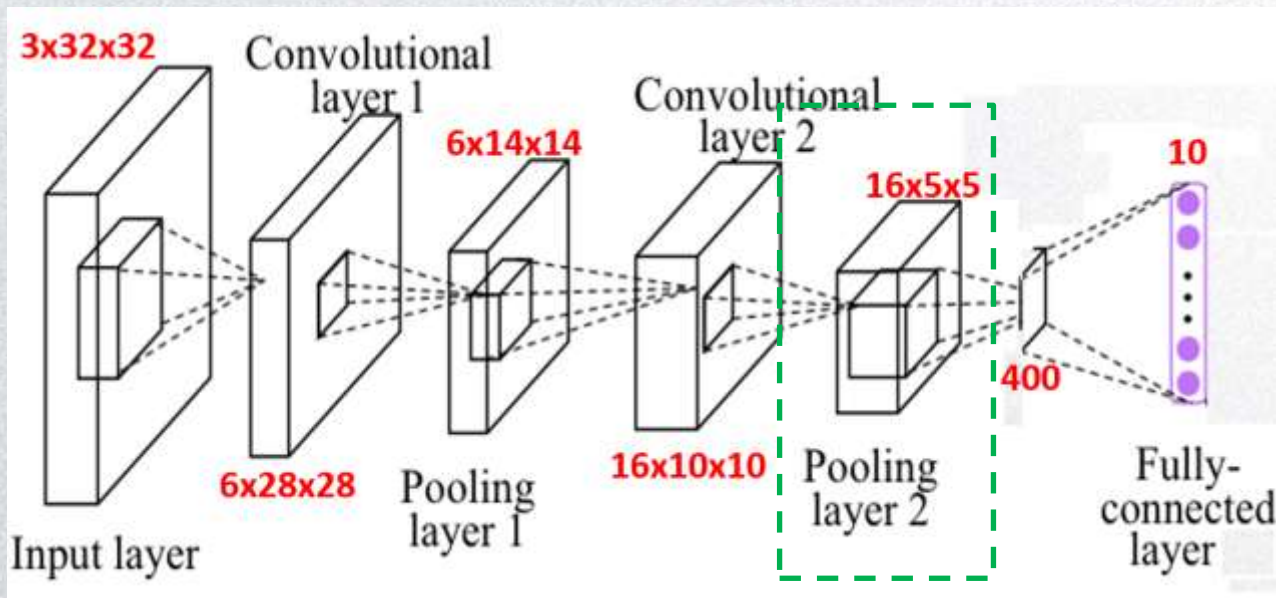
- 第二层卷积，16个5*5的卷积核

```
20 # Conv-2
21 Conv2 = nn.Conv2d(in_channels=6,out_channels=16,kernel_size=5,stride=1,padding=0)
22 Nf2 = nn.ReLU()
23 output2 = Nf2(Conv2(output1))
24 print(output2.shape)
```

```
torch.Size([1, 16, 10, 10])
```

卷积神经网络

- 最大池化层, $f=2, s=2$, 最后输出为 $16 \times 5 \times 5$, 标记为POOL2
- 第二个卷积层和最大池化层组成**Layer2**



每个数值都代表了它从属于的那个数字的权重



练习: POOL2

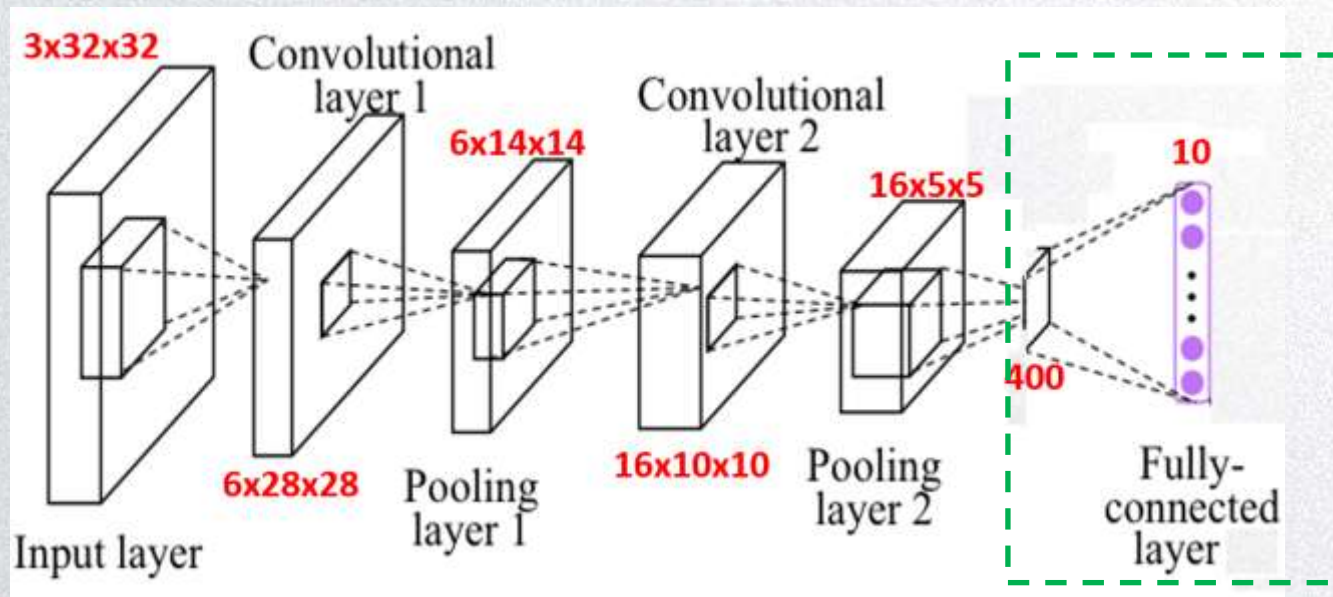
- POOL2

```
25 # Pool-2
26 Pool2 = nn.MaxPool2d((2,2),stride=2,padding=0)
27 output2 = Pool2(output2)
28 print(output2.shape)
```

```
torch.Size([1, 16, 5, 5])
```

卷积神经网络

- $16 \times 5 \times 5$ 矩阵包含 400 个元素，平整化为一个大小为 400 的一维向量
- 400 个单元作为输入，10 个单元作为输出，构建全连接层 (FC)





卷积神经网络

- $16 \times 5 \times 5$ 矩阵包含400个元素，平整化为一个大小为400的一维向量
- 400个单元作为输入，10个单元作为输出，构建全连接层 (FC)
- “全连接”：维度为 400×10 ，这400个单元与这10个单元的每一项连接，有10个输出；对输出做SoftMax，得到属于各个数字的概率

```
29 # Flatten
30 output3 = output2.view(1, -1)
31 print(output3.shape)
32 # FC
33 FC = nn.Linear(400, 10)
```

```
torch.Size([1, 400])
torch.Size([1, 10])
tensor([[0.1031, 0.0958, 0.1121, 0.0933, 0.1066, 0.0989, 0.1012, 0.0872, 0.1061,
         0.0958]], grad_fn=<SoftmaxBackward>)
```

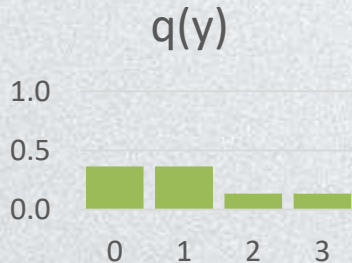
```
37 output = torch.softmax(output3, dim=1)
38 print(output)
```


神经网络的输出层

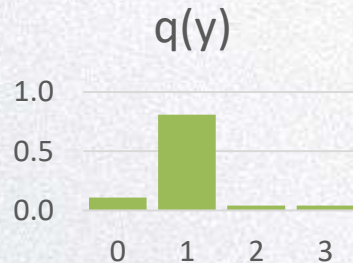
- 多分类： 假设总共有C类, $y \in \{0, 1, 2, \dots, C - 1\}$
 - 模型输出一个C维向量 $\mathbf{z} = [z[0], z[1], \dots, z[C - 1]] \in R^C$
 - 利用softmax函数计算: 能够把任意数列输出为一个概率分布

$$q(y = i) = \frac{e^{z[i]}}{\sum_j e^{z[j]}}$$

y	z	exp(z)	q(y)
0	1.000	2.718	0.366
1	1.000	2.718	0.366
2	0.000	1.000	0.134
3	0.000	1.000	0.134



y	z	exp(z)	q(y)
0	1.000	2.718	0.110
1	3.000	20.086	0.810
2	0.000	1.000	0.040
3	0.000	1.000	0.040





nn.Softmax



Applies the Softmax function to an n-dimensional input Tensor rescaling them so that the elements of the n-dimensional output Tensor lie in the range [0,1] and sum to 1.

Softmax is defined as:

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

When the input Tensor is a sparse tensor then the unspecified values are treated as `-inf`.

Shape:

- Input: (*) where * means, any number of additional dimensions
- Output: (*), same shape as the input

```
>>> m = nn.Softmax(dim=1)
>>> input = torch.randn(2, 3)
>>> output = m(input)
```

```
>>> input
tensor([[ -0.4652,  1.0498,  0.6427],
        [ 1.3657, -1.4031,  0.6516]])
```

dim=1

```
>>> output
tensor([[0.1166, 0.5304, 0.3530],
        [0.6442, 0.0404, 0.3154]])
```

dim=0

```
>>> output
tensor([[0.1381, 0.9208, 0.4978],
        [0.8619, 0.0792, 0.5022]])
```



卷积神经网络

- 随着神经网络深度的加深，高度和宽度通常都会减少（从 32×32 到 28×28 ，到 14×14 ，到 10×10 ，再到 5×5 ），而通道数量会增加（从3到6到16不断增加），然后使用一个全连接层。
- 在神经网络中，另一种常见模式就是一个或多个卷积后面跟随一个池化层，然后一个或多个卷积层后面再跟一个池化层，然后是几个全连接层，最后是一个softmax。

搭建卷积神经网络



```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=6, kernel_size=5, stride=1, padding=0)
        self.pool1 = nn.MaxPool2d((2, 2), stride=2, padding=0)
        self.conv2 = nn.Conv2d(in_channels=6, out_channels=16, kernel_size=5, stride=1, padding=0)
        self.pool2 = nn.MaxPool2d((2, 2), stride=2, padding=0)
        self.fc = nn.Linear(400, 10)
        self.nf = nn.ReLU()

    def forward(self, x):
        output1 = self.nf(self.conv1(x))
        print(output1.shape)
        output1 = self.pool1(output1)
        print(output1.shape)
        output2 = self.nf(self.conv2(output1))
        print(output2.shape)
        output2 = self.pool2(output2)
        print(output2.shape)
        # Flatten
        output3 = output2.view(1, -1)
        print(output3.shape)
        # FC
        output3 = self.fc(output3)
        print(output3.shape)
        # SoftMax
        output = torch.softmax(output3, dim=1)
        return output
```



搭建卷积神经网络

- 前向计算

```
image = Image.open('5.jpg').convert('RGB')
input = transforms.ToTensor()(image).unsqueeze(0)
print(input.shape)

model = CNN()
output = model(input)
print(output)
```



搭建卷积神经网络

- 前向计算, `batchsize > 1` 这种情况下view的时候不能直接写成1了

```
image1 = Image.open('5.jpg').convert('RGB')
input1 = transforms.ToTensor()(image1).unsqueeze(0)
image2 = Image.open('3.jpg').convert('RGB')
input2 = transforms.ToTensor()(image2).unsqueeze(0)
input = torch.cat([input1, input2], 0)
print(input.shape)

model = CNN()
output = model(input)
print(output.shape)
print(output)
```


练习



- $3 \times 32 \times 32$ 的图片，第一层用6个大小为 5×5 的滤波器，这一层有多少个参数？
- 不论输入图片有多大，参数始终都是456个。即使这些图片很大，参数却很少，这就是卷积神经网络的一个特征，可以“避免过拟合”。
- 已经知道如何提取6个特征，可以应用到大图片中，而参数数量固定不变，此例中提取一种特征只需要75个参数，相对较少。



为什么使用卷积

- 参数数量
- $3 \times 32 \times 32$ 的图片，假设用6个大小为 5×5 的滤波器，输出维度为 $6 \times 28 \times 28$ ，每个滤波器有 $5 \times 5 \times 3 + 1 = 76$ 个参数(加上bias)，共 $76 \times 6 = 456$ 个参数。
- 如果用全连接层，输入 $3 \times 32 \times 32 = 3072$ 个节点，输出 $6 \times 28 \times 28 = 4704$ 个节点，参数数量 $4074 \times 3072 \approx 1400$ 万
- 如果图片大小为 1000×1000 ，全连接权重矩阵会变得非常大。卷积参数数量保持不变。

为什么使用卷积

- 参数共享
- 特征检测如垂直边缘检测如果适用于图片的某个区域，那么它也可能适用于图片的其他区域。每个特征检测器都可以在输入图片的不同区域中使用同样的参数。

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



*

1	0	-1
1	0	-1
1	0	-1



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0



为什么使用卷积

- 稀疏连接
- 例：0是通过 3×3 的卷积计算得到的，它只依赖于这个 3×3 的输入的单单元格，右边这个输出单元（元素0）仅与36个输入特征中9个相连接；其它像素值都不会对输出产生任何影响。

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0



*

1	0	-1
1	0	-1
1	0	-1



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0





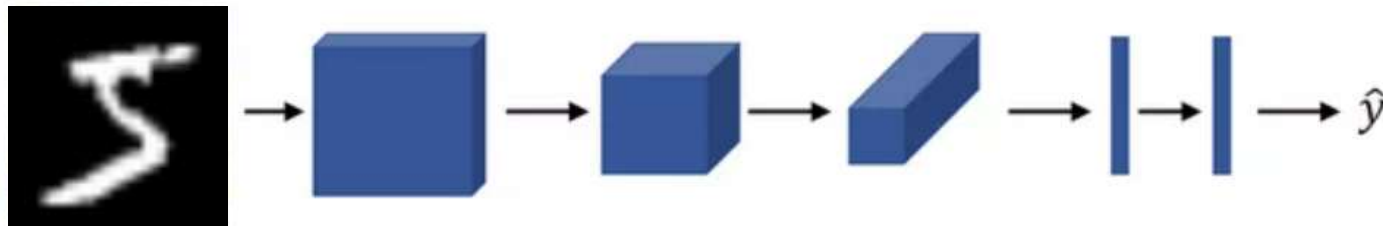
为什么使用卷积

- 卷积神经网络通过参数共享和稀疏连接这两种机制减少参数，以便使用更少的训练数据来训练，防止过拟合。
- 平移不变性：向右移动两个像素，图片中的猫依然清晰可见；因为神经网络的卷积结构使得即使移动几个像素，这张图片依然具有非常相似的特征。

卷积神经网络训练

- 训练集: x 表示一张图片, y 是类别标注
- 损失函数: 如果使用交叉熵损失函数, 不需要做softmax
- 优化器: 例如梯度下降法或Adam

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

池化的时候会有无法交互得到的两个区域
但是在后面的层数中可以交互得到



谢谢！

